From Search Dynamics to Semi-Ideal Computers

Cumulative Verified Histories, Rationally Weighted Theorem Mazes, and Reflective ATPs

A General Framework for ATPs, ALDs, and AMLDs, with DATA as a Distinguished Case

Brian Tenneson

August 14, 2026 – Version 0.2.1

Suggested arXiv classifications: primary `cs.LO`; secondary `cs.AI`; cross-list candidate `math.CO`.

Abstract

This sequel begins with the graph-theoretic picture: theorem proving is treated as maze solving. Proof states are vertices, legal inference applications are directed edges, the initial hypotheses/axioms form the entrance, and states containing a verifier-accepted target certificate are exits. Unit weighting $w(e) = 1$ recovers ordinary shortest-proof breadth-first search; exact nonnegative rational weights lead to weighted theorem mazes for which classical shortest-path methods can be imported whenever their hypotheses are satisfied. A short maze primer therefore precedes the logical and dynamical machinery and records the usual worst-case escape times for BFS, DFS, Dijkstra, A*, DAG shortest paths, and all-pairs methods on explicit graphs.

The crucial distinction is between an explicit maze and an implicit theorem maze. Classical shortest-path problems are polynomial in the size of the graph that has already been presented, but the number of proof states generated by a compact logical problem may itself be enormous. The central research objective is therefore not to claim that rational weighting makes theorem proving easy; it is to reduce the fraction of the implicit maze that must be exposed. The paper ranks quotient and partial-order reductions, admissible abstractions combined with Proof-Horizon incumbent bounds, conservative compilation and verified landmarks, and min-plus path algebra ahead of more speculative navigation layers when exact reduction is available.

The manuscript then lifts staged verifier-backed search dynamics to semi-ideal computers (SICs), ATPs, automated logical deciders (ALDs), and automated metalogical deciders (AMLDs). Version 0.2.1 adds a control-AMLD layer in which proof, refutation, independence, and meta-level search are treated as feedback-controlled actions. Drift, variance, control-Lyapunov margins, Bellman values, and concave resource allocation provide measurable control quantities without giving learned components certificate authority. The update also incorporates the settlement-compass experiment sequence through Experiments 004 and 005. The clean nested-shell/MAX diagnostic shows early saturation of global discrimination but continued improvement in local direct-parent navigation; illustrative fits place the global AUC scale far below the top-k scale. The first retrospective trajectory diagnostics then show why static ranking quality is insufficient for closed-loop settlement: positive aggregate learned drift can coexist with controller failure through off-DAG successor choices. These results are reported as empirical diagnostics and research guidance, not as an end-to-end ATP speedup claim.

Finally, absorbing settlement connects halting to fixed points, while IFS, semigroup, continuous, and fractional constructions remain optional control layers rather than part of the definition of rational edge weight. A sufficiently expressive induced ATP may reason about a frozen code of a prover or controller, subject to ordinary incompleteness and self-reference limits. DATA is treated as a distinguished implementation rather than as the general definition.

Contents

1 Big picture: shrink the theorem maze before trying to outrun it

Research judgment, not a theorem. The ranking in this section is a research-priority assessment. It is based on which ideas attack the explicit bottleneck identified by the maze formulation—state-space explosion—while preserving exact logical correctness. Classical graph search already solves an explicitly presented finite maze efficiently relative to the size of that maze. The hard ATP problem is that the theorem maze is normally implicit and may be enormous before a useful exit is exposed. The most promising theories are therefore the ones that can provably or measurably reduce the state space, supply exact lower or upper bounds, or reuse verified structure before search expands most of the maze [14, 15, 16].

1.1 The program in one diagram

The proposed near-term hierarchy is

$$\begin{array}{c}\text{formal problem } (F, \Gamma, \varphi)\\ \downarrow\\ \text{implicit theorem maze } G\\ \downarrow \quad \text{exact quotient / partial-order / symmetry reduction}\\ G/\!\sim\\ \downarrow \quad \text{admissible abstraction and verified landmarks}\\ \bar{G}\\ \downarrow \quad \text{lower bound } h + \text{verified incumbent } B\\ \text{incumbent-admissible search region}\\ \downarrow\\ \text{Dijkstra / A* / branch-and-bound / learned ordering}\\ \downarrow\\ \text{verifier-accepted exit.}\end{array}$$

The central aim is not to make every proof short. It is to arrange that the machine never materializes large families of routes that are redundant for the declared objective.

1.2 Most promising theories

1.3 The strongest near-term synthesis: quotient, abstraction, and bounds

The most concrete mathematical target is to combine exact state reduction with an admissible abstract distance. Let

$$G = (V, E, s, Z, w)$$

be a nonnegatively weighted theorem maze. First choose a verifier-respecting equivalence relation $\sim$ that merges only states whose distinction is irrelevant to the reachability and cost objective being preserved. Standard symmetry and partial-order reductions show how dramatic such reductions can be in other reachability systems, but the proof-specific preservation conditions must be proved rather than assumed [8, 9, 10].

Next let

$$\pi : V \to \bar{V}$$

Priority	Theory or method	Why it looks promising for ATP search
1	Quotient state spaces, symmetry reduction, partial-order reduction, proof-DAG canonicalization	Directly attacks state explosion by identifying histories or interleavings that need not be explored separately. Model checking developed quotient, symmetry, and partial-order methods precisely because explicit reachability graphs explode; the proof-search analogue is to preserve theorem reachability and the chosen cost while exploring fewer representatives [8, 9, 10, 14].
2	Admissible abstraction + A* / branch-and-bound + Proof Horizon	Gives a mathematically safe way to combine a lower bound on cost-to-go with an upper bound from any verified incumbent. This can remove whole regions without trusting a learned guess. Pattern-database and abstraction heuristics show how abstract state spaces can generate admissible lower bounds in other search domains [3, 11, 15, 16].
3	Conservative compilation, verified shared landmarks, dynamic programming on proof DAGs	Reuses repeated verified substructure instead of rediscovering it. This is especially attractive when many routes share the same lemmas or subproofs; the author's earlier compilation and bank results give theorem-search-specific motivation [14, 19].
4	Min-plus / tropical path algebra over exact rational costs	Gives a clean algebraic home for weighted theorem mazes: sequential inference costs add and alternative proof routes are minimized. It is more a unifying theory than a guaranteed speedup, but it may make dynamic programming, composition, and quotient costs easier to state and analyze [12].
5	Learned cost-to-go, portfolio scheduling, Hilbert/locality ordering	Potentially powerful for deciding which surviving state to expand first. These methods can improve navigation without changing the verifier, but their advantage is empirical and distribution-dependent; Hilbert ordering in particular should be treated as a benchmarked locality heuristic rather than an optimality theorem [18, 19].
6	IFS/semigroup, fractional iteration, reflective meta-ATP	Mathematically interesting long-term control theories. They may eventually describe hitting-time geometry or prove bounded facts about the searcher, but they are not prerequisites for the maze reduction program and currently have a less direct path to speed than exact quotienting and admissible bounds [19, 14].

Table 1: Research-priority assessment for the maze-first program. The ordering is a judgment about likely leverage, not a proved dominance theorem.

map the concrete maze into an abstract or relaxed maze $\bar{G} = (\bar{V}, \bar{E}, \bar{Z}, \bar{w})$. Assume that every concrete path from v to an exit maps to an abstract path from $\pi(v)$ to $\bar{Z}$ of no greater cost.

Proposition 1.1 (Abstract-distance lower bound)**.** *Under the preceding path-relaxation hypothesis,*

$$h(v) := d_{\bar{w}}(\pi(v), \bar{Z})$$

is admissible for the concrete theorem maze:

$$h(v) \leq d_w(v, Z).$$

Proof. Every concrete v-to-exit path has an abstract image of no greater cost. Taking the minimum or infimum over abstract paths therefore cannot exceed the minimum or infimum over concrete paths. □

Now suppose the fast search wing has already found and verified an exit route of cost B. For a reached state v, let $g(v)$ be the exact cost already spent. Any route improving the incumbent must

satisfy

$$g(v) + h(v) < B.$$

Therefore the next search can be confined to the *incumbent-admissible region*

$$\boxed{\mathcal{C}_B(h) = \{v \in V : g(v) + h(v) < B\}.}$$

This is the clearest current mathematical answer to the question "where should the ATP not look?" The incumbent supplies an exact upper bound; the abstraction supplies an exact lower bound; quotienting reduces duplicate states before either bound is applied. The tighter the valid quotient and the lower bound, the smaller the region that remains eligible for an improving proof [3, 11, 15].

Caution 1.2 (Keep the cost metric fixed). The same metric must underlie g, h, and B. A native proof-length Horizon cannot be combined without proof with a heuristic that lower-bounds wall-clock search cost, and a macro-edge search cost need not equal the cost of its transparent native expansion. This metric-separation issue is already explicit in the DATA/Horizon framework [19].

1.4 A natural algebra for proof classes: min-plus

The weighted maze and the proof-tour quotient fit the min-plus (tropical) semiring

$$\mathbb{T}_{\mathbb{Q}} = (\mathbb{Q}_{\geq 0} \cup \{\infty\}, \oplus, \otimes), \qquad a \oplus b = \min(a, b), \quad a \otimes b = a + b.$$

Along one proof route, edge costs compose by $+$; among alternative routes proving the same theorem, the optimal representative is selected by min. On a finite or otherwise effectively proper layer where the minimum is attained, the weighted Proof Horizon can therefore be written schematically as

$$\boxed{H_w(\Gamma, \varphi) = \bigoplus_{P \in [\varphi]} \bigotimes_{e \in P} w(e).}$$

This does not make state explosion disappear. Its value is structural: it turns "proofs are paths and the Horizon is the cheapest representative" into an ordinary algebraic-path statement, connecting the theorem-maze program to the established theory of graphs over semirings and dioids [12].

1.5 What should be tested first

The first serious implementation target should therefore be a matched benchmark in which the same verifier-backed theorem mazes are searched under progressively stronger but separately measured reductions:

raw BFS/Dijkstra
↓
canonical proof states
↓
partial-order or quotient reduction
↓
verified landmarks/macros
↓
admissible abstraction
↓
incumbent-bounded A* or branch-and-bound.

Learned ranking and Hilbert ordering can then be layered on the remaining region and evaluated by ablation. Fractional/continuous dynamics should be tested later, after the simpler graph reductions have been exhausted. This ordering makes failures informative: if a speedup occurs, the experiment can identify whether it came from structural compression, exact bounds, or heuristic navigation rather than attributing everything to one large architecture.

2 Maze theory first: the minimum needed for theorem search

The main organizing idea of this sequel is deliberately elementary: before discussing SICs, fixed points, IFS language, or reflective control, treat theorem proving as maze solving. A maze is a graph-search problem. The classical shortest-path literature then tells us what can be solved efficiently once the graph is explicitly available, while the theorem-proving difficulty is pushed into the size and structure of the implicit graph that must be generated. Moore's early maze formulation, Dijkstra's weighted shortest-path algorithm, and the later A* theory are the basic classical reference points [1, 2, 3, 4].

2.1 Mazes as directed graphs

For present purposes a maze is a directed graph with an entrance and one or more exits,

$$\mathcal{M} = (V, E, s, Z),$$

where V is the vertex set, $E \subseteq V \times V$ is the directed edge set, $s \in V$ is the entrance, and $Z \subseteq V$ is the exit set. An edge $u \to v$ means that moving from u to v is legal. A finite directed walk is a sequence of legal edges; a path is a walk with no repeated vertices. In this manuscript the informal word "tour" means a finite directed walk from the entrance toward an exit, not necessarily a closed tour.

A graph is *sparse* when $|E|$ is of the same order as $|V|$ and *dense* when $|E| = \Theta(|V|^2)$. This distinction matters because an $O(|V| + |E|)$ algorithm is linear in the explicit graph representation but becomes $O(|V|^2)$ on a dense maze [4].

2.2 Unweighted and weighted mazes

In an unweighted maze every edge may be assigned unit cost,

$$w(e) = 1.$$

Then the cost of a path is simply its number of edges. Breadth-first search (BFS) therefore finds a route with the minimum number of edges from the entrance to an exit [1, 4].

A weighted maze adds a cost map

$$w : E \to \mathbb{R}.$$

This paper restricts the exact ATP cost layer to

$$w : E \to \mathbb{Q}_{\geq 0}.$$

The restriction to exact nonnegative rational numbers is intentional. Rational weights can be represented by finite numerator/denominator pairs, added exactly, and compared exactly. Thus an optimality statement need not depend on floating-point roundoff. Dijkstra's algorithm applies to nonnegative weights, and A* adds an admissible estimate of the remaining cost [2, 3, 4].

The path cost is

$$w(P) = \sum_{e \in P} w(e).$$

The shortest escape cost is

$$d_w(s, Z) = \inf\{w(P) : P \text{ is a directed path from } s \text{ to some } z \in Z\}.$$

For a finite maze with nonnegative weights and a reachable exit, the infimum is attained by a simple path.

2.3 Effective naming of rational edge weights

The author's earlier closed-form surjection

$$r : \mathbb{N} \twoheadrightarrow \mathbb{Q}$$

provides an explicit natural-number naming scheme for rational values [13]. Taking

$$r_+(n) = |r(n)|$$

gives a surjection $r_+ : \mathbb{N} \twoheadrightarrow \mathbb{Q}_{\geq 0}$. Hence a theorem-maze edge may store a natural code k_e while its exact declared cost is

$$w(e) = r_+(k_e).$$

This coding is not needed for Dijkstra or A*—ordinary reduced numerator/denominator pairs already suffice—but it is useful conceptually and computationally when one wants every exact rational cost to have a discrete address. It also keeps two roles separate: rational codes name edge costs, while Hilbert-style coordinates and orderings may later be used to name or prioritize regions of the state space [18].

2.4 What "escaping the maze" normally costs

Let $n = |V|$ and $m = |E|$. The following are the standard worst-case graph-operation bounds needed later. They describe an *explicitly represented* finite maze. Exact rational arithmetic adds the cost of exact integer addition and comparison; the table suppresses that bit-complexity factor and records the usual graph-search bounds [4, 2, 3, 6].

The table gives an important perspective. Rational edge weights do not by themselves make shortest-path search intractable. For a dense finite maze that has already been written down, BFS, Dijkstra with a dense implementation, and DAG shortest-path dynamic programming all have familiar $O(n^2)$ graph-operation bounds. The difficulty for ATP is that n itself may be exponential or worse as a function of a compact formal problem description. A procedure polynomial in the number of explicit proof states can therefore still be impractical in the original theorem-proving input size. This explicit-versus-implicit distinction is central to the search-reduction program developed below.

2.5 Implicit mazes and the usual exponential search bound

The same point is often expressed in AI search notation. Suppose the maze is not given in advance but is generated as a search tree with effective branching factor b, the shallowest exit is at depth d, and a depth-first search may descend to depth ℓ. In the usual tree-search accounting, BFS can require $O(b^d)$ generated nodes and $O(b^d)$ memory; DFS can require $O(b^\ell)$ time while using only linear-in-depth stack memory up to the cost of storing successor information. A* has no

Method	Guarantee	General explicit-graph worst case	Dense maze $m = \Theta(n^2)$
DFS	Finds some reachable exit; not generally shortest	$O(n+m)$	$O(n^2)$
BFS	Minimum number of edges when all costs are 1	$O(n+m)$	$O(n^2)$
Dijkstra	Minimum total cost for nonnegative weights	$O((n+m)\log n)$ with a binary heap; $O(n^2)$ with a simple array implementation	$O(n^2)$ is standard and natural for dense graphs
A*	Minimum cost under the usual admissibility/implementation hypotheses	Worst case may search essentially the same region as Dijkstra; $h = 0$ reduces to Dijkstra	No worst-case asymptotic miracle follows from heuristic search alone
DAG shortest path	Minimum total cost after a topological ordering	$O(n+m)$	$O(n^2)$
Floyd–Warshall	All-pairs shortest-path distances	$O(n^3)$	$O(n^3)$

Table 2: Usual worst-case escape/shortest-path bounds for an explicit finite maze. The ATP difficulty discussed below is that the theorem maze is normally implicit and can be enormously larger than its logical input description.

general worst-case polynomial-in-d guarantee: with an uninformative heuristic it degenerates toward uniform-cost/Dijkstra-style expansion, and standard worst cases remain exponential in solution depth [5, 3]. These bounds are not in conflict with the $O(n+m)$ or $O(n^2)$ explicit-graph bounds above: in an implicit maze, the number n of states that would have been written down can itself be on the order of b^d.

For theorem proving this is the more revealing worst case. The classical graph algorithm may be polynomial in the maze it sees, while generating enough of that maze to reach the theorem can still consume exponential resources. The central design problem is therefore to reduce the effective branching factor, identify equivalences, reuse landmarks, improve lower bounds, or otherwise avoid materializing most of the search tree.

2.6 From an ordinary maze to a theorem maze

Fix a formal system F, a set of hypotheses and/or available axioms Γ, and a target WFF φ. Following the theorem-graph program developed in the author's earlier work, use *proof states* rather than individual formulas as vertices [14]. A proof state is a cumulative set p of formulas already available, with

$$\Gamma \subseteq p.$$

The entrance is

$$s = \Gamma.$$

The exit set is

$$Z_\varphi = \{p : \varphi \in p\}.$$

Thus an ATP succeeds exactly when it reaches a proof state containing the target.

The state formulation also resolves the arity issue for inference rules. Suppose r is a k-ary inference rule and

$$a_1, \ldots, a_k \in p$$

are premises already present in the current state. If a legal application of r derives β, place the directed edge

$$p \xrightarrow{r(a_1,\ldots,a_k)} p \cup \{\beta\}.$$

A binary or k-ary inference therefore remains a single ordinary graph edge because all premises are already contained in the source proof state. If one instead insists that vertices are individual WFFs, the natural object is a directed hypergraph or a bipartite formula/inference graph. For algorithmic purposes, the proof-state graph keeps the representation within ordinary directed graph theory [14].

A proof of φ is therefore the pullback of a successful directed route

$$\Gamma \rightsquigarrow Z_\varphi.$$

The verifier checks that every edge label is a legitimate inference application. Graph search chooses the route; logic decides whether each move is legal.

2.7 Unit normalization and the BFS baseline

The clean baseline is

$$\boxed{w(e) = 1 \quad \text{for every legal inference edge.}}$$

Then a proof route $P = (e_1, \ldots, e_L)$ has cost

$$w(P) = \sum_{i=1}^{L} 1 = L.$$

Consequently the theorem-maze distance is exactly shortest proof length,

$$d_1(\Gamma, Z_\varphi) = \min\{|P| : P \text{ proves } \varphi\}.$$

This is the graph-theoretic form of the author's Proof-Horizon/BFS viewpoint: unit-weight breadth-first search reaches a target first at the minimum inference depth [14, 15]. Choosing the unit weight 1 is a normalization: one does not need to carry a separate physical unit through every formula. One edge simply means one unit of declared proof cost.

2.8 Rational weights are costs, not fractional inferences

After the unit case is understood, one may declare different exact rational costs,

$$w(e) \in \mathbb{Q}_{\geq 0}.$$

A weight such as $3/7$ means that the edge costs $3/7$ of the chosen cost unit. It does *not* by itself mean that the inference is the fractional iterate $T^{3/7}$ of a logical transformation. Fractional iteration is a separate dynamical question requiring a semigroup or embedding law such as

$$\Phi_{s+t} = \Phi_s \circ \Phi_t.$$

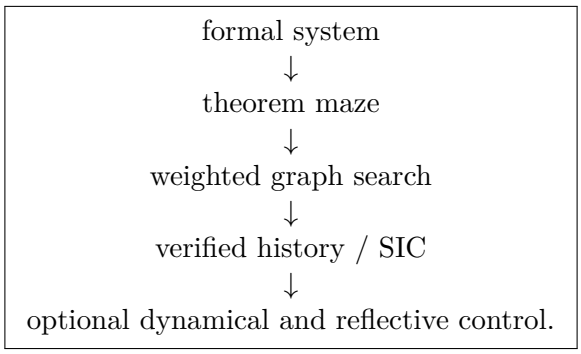
The weighted-maze layer should therefore be established independently. Continuous or fractional search dynamics may later be placed over the maze as an optional model of navigation, but rational edge length alone does not imply fractional inference.

2.9 Where geometry and Hilbert ordering enter

The author's Hilbert theorem-search work proposes using space-filling-curve orderings as a navigation device after proof/search states have been embedded into a finite-dimensional feature representation [18]. In the maze language this is best viewed as an ordering policy for which region or frontier point to inspect next. It does not alter the legal edges, the verifier, or the edge costs. The classical shortest-path guarantees remain attached to BFS, Dijkstra, A*, DAG methods, and related graph algorithms; Hilbert ordering is a candidate locality-preserving heuristic to be benchmarked against those baselines.

2.10 The big picture before the rest of the paper

The sequel will therefore use the hierarchy



The first computational question is not whether a fixed point or proof exists in the abstract, but how much of the implicit maze must be generated and searched before a verifier-accepted exit is reached. The remaining sections study exact reductions of that search region, quotient ideas, incumbent bounds, learned navigation, and only afterward the broader SIC/IFS/reflective interpretation.

3 Theorem proving as rationally weighted maze solving

The graph-theoretic viewpoint from *Depths of a Simulation* can be strengthened into a transfer principle. Proof search is treated as directed graph traversal: proof states are vertices, legal inference transactions are edges, and target states are exits [14].

Definition 3.1 (Rationally weighted theorem maze)**.** *A rationally weighted theorem maze is*

$$\mathcal{G} = (V, E, s, Z, w)$$

where (V, E) is a directed graph, $s \in V$ is the initial proof/search state, $Z \subseteq V$ is the set of certificate-bearing exits, and

$$w : E \to \mathbb{Q}_{\geq 0}$$

assigns each directed edge an exact nonnegative rational cost. The cost of a finite path $P = (e_1, \ldots, e_m)$ is

$$w(P) = \sum_{i=1}^{m} w(e_i) \in \mathbb{Q}_{\geq 0}.$$

Rational weights are a deliberate restriction. They can be stored, added, and compared exactly. This avoids making a logical optimality theorem depend on floating-point ordering accidents. Integer transaction counts are the special case $w(e) \in \mathbb{N}$.

Theorem 3.2 (Rational-Weighted Maze Transfer Principle)**.** *Let $\mathcal{A}$ be a maze-search algorithm whose correctness theorem holds for a class $\mathscr{C}$ of effectively presented directed graphs with nonnegative rational edge weights. Suppose a theorem-proving or settlement problem can be encoded as a member of $\mathscr{C}$ so that:*

(i) vertices encode admissible proof/search states;

(ii) directed edges encode admissible transactions or inference steps;

(iii) target vertices are exactly states containing a verifier-accepted certificate of the requested type; and

(iv) path weight is the declared ATP search cost.

Then applying $\mathcal{A}$ to the encoded theorem maze yields an ATP/ALD/AMLD search policy with the same route-finding and path-cost guarantee, subject to the same graph-theoretic hypotheses.

Proof. The encoding is a cost-preserving correspondence between maze paths and admissible search paths. Therefore any path returned by $\mathcal{A}$ pulls back to an admissible ATP path, and any optimality guarantee over the maze path weights transfers to the declared ATP search cost. Logical acceptance still requires the independent certificate verifier. □

Remark 3.3 (Examples)*.* For finite nonnegative rationally weighted theorem mazes, Dijkstra's algorithm gives a minimum-cost route. A* gives the same conclusion under the usual admissibility conditions, with the standard implementation qualifications concerning consistency and node reopening. On a finite DAG, dynamic programming after a topological ordering gives minimum-cost routes. These are not new graph theorems; the contribution is the exact pullback to verifier-backed proof or settlement search.

Caution 3.4 (Search cost versus native proof cost)*.* A weighted theorem maze may price search transactions, macro rules, verifier calls, or other operations. That metric need not equal native proof length. If a banked lemma is transparently expanded before native proof cost is measured, the bank may dramatically reduce search cost while leaving the native minimum proof Horizon unchanged [19]. Every reported shortest-path claim must therefore name the metric and presentation.

4 Search-space reduction before learning

The graph interpretation provides several exact or theorem-backed reductions before any learned heuristic is trusted.

4.1 Cumulative verified-history collapse

Many transient trajectories can lead to the same permanent verified ledger. If the objective is mathematical settlement rather than replaying every heuristic choice, states that differ only in irrelevant transient history can often share downstream mathematical information. Care is required because the cumulative history alone need not determine future controller behavior, but it is already a major conceptual reduction in what must be remembered as mathematical progress.

4.2 Novelty DAG

In the novelty state theorem graph, retain only nontrivial transitions that add new verified information. Strict inclusion then rules out directed cycles, producing a DAG [14]. Identity moves, rediscovery loops, and reversible syntactic wandering can be suppressed or represented outside the verified-state graph.

4.3 Quotienting by verified equivalence

When the formal system supplies a sound equivalence relation and the search objective is invariant under it, equivalent presentations can be quotiented or canonically oriented. This is a standard graph reduction, but in ATP design it can eliminate large families of syntactic duplicates. The quotient must be verifier-respecting; heuristic similarity is not enough.

4.4 Conservative compilation and macro edges

Depths of a Simulation proves a conservative compilation result for finite provable workloads: verified derivations can be packaged as sound derived rules, lowering target transaction counts without changing theoremhood [14]. In graph language, a previously expensive verified path is replaced by a reusable macro edge. This is the mathematical justification for viewing a theorem library as a compression layer over repeated proof structure.

4.5 Shared verified landmarks

A shared bank can turn repeated rediscovery into one verified discovery followed by cheaper retrieval. DATA 3.0 gives a conditional repeated-substructure savings theorem and separately shows that transparent lemma reuse need not change the native proof Horizon [19]. For search, the bank changes the geometry; for native minimum proof cost, it need not.

4.6 Incumbent bounds

The Proof-Horizon program supplies an especially useful reduction. Once any valid proof or settlement certificate of cost B is verified, the unknown optimum c^* satisfies

$$c^* \leq B.$$

For minimum-cost certification, all candidates with cost $\geq B$ are irrelevant to the question whether a cheaper certificate exists. Each improved incumbent shrinks the remaining bounded region [15, 16].

Proposition 4.1 (Rational branch-and-bound pruning)**.** *Let $\mathcal{G}$ be a theorem maze with nonnegative rational edge weights. Let B be the cost of a verified incumbent route to an exit. For a reached vertex v, let $g(v)$ be the exact cost already incurred and let $h(v) \in \mathbb{Q}_{\geq 0}$ be a valid lower bound on the cost of every continuation from v to an exit. If*

$$g(v) + h(v) \geq B,$$

then no route through v can improve the incumbent, so v may be pruned from a search whose sole objective is to find a route of cost $< B$.

Proof. Every continuation through v has total cost at least $g(v) + h(v) \geq B$. □

This proposition combines the Horizon incumbent idea with ordinary weighted-maze branch-and-bound. Even the trivial lower bound $h = 0$ removes every partial path whose cost has already reached B; stronger admissible lower bounds remove more.

5 Heuristic branches and where to look

The Branch-Covering Theorem in *Depths* says, roughly, that a proof-covering nondeterministic policy contains a branch able to shadow a shortest proof; heuristics then face the navigation problem of allocating enough computation to such branches [14]. This does not identify the good branch, but it transforms the intelligence problem into ranking, covering, diversification, and resource allocation.

For a theorem maze, a learned critic can estimate quantities such as

$$\widehat{d}(v, Z), \qquad \widehat{P}(\text{settlement within budget} \mid v), \qquad \widehat{Q}(v, a),$$

without ever acquiring proof authority. The verifier decides acceptance. The learned model decides where to spend the next unit of search.

A practical architecture can combine exact and heuristic components:

$$\begin{aligned}&\text{exact rational costs} + \text{incumbent bound} + \text{admissible lower bounds}\\ &+ \text{learned ranking} + \text{portfolio diversity} + \text{verified bank}.\end{aligned}$$

The exact pieces prune safely; the learned pieces reorder or allocate the remaining work.

6 The theoremhood quotient: proofs as representatives

The maze picture suggests a deliberately coarse equivalence relation on successful routes. Fix a formal system F, hypotheses Γ, target theorem φ, and a theorem maze whose edge labels record the inference applications used along a directed walk. Let $\Pi(P)$ denote the pullback of a finite proof tour P to its corresponding finite sequence of well-formed formulas (WFFs). Let $\mathcal{W}_\varphi$ be the set of finite directed tours from the entrance to a designated φ-exit, and let

$$\mathcal{P}_\varphi = \{P \in \mathcal{W}_\varphi : \Pi(P) \text{ is a verifier-accepted proof of } \varphi \text{ from } \Gamma\}.$$

Here "tour" means a finite directed walk from the entrance to an exit; no closed-walk convention is intended.

Definition 6.1 (Logical proof-tour relation)**.** *For $P, Q \in \mathcal{W}_\varphi$, write*

$$P \sim_\varphi Q \quad \Longleftrightarrow \quad \Pi(P) \text{ and } \Pi(Q) \text{ are both proofs of } \varphi \text{ from } \Gamma.$$

Theorem 6.2 (Proof-Tour Equivalence Theorem)**.** *Restricted to the successful proof-tour set $\mathcal{P}_\varphi$, the relation $\sim_\varphi$ is an equivalence relation. Equivalently, for any $P, Q \in \mathcal{P}_\varphi$,*

$$P \sim_\varphi Q \quad \Longleftrightarrow \quad \Pi(P) \text{ and } \Pi(Q) \text{ are both proofs of } \varphi \text{ from } \Gamma.$$

If $\mathcal{P}_\varphi \neq \varnothing$, then every two successful tours are equivalent and therefore

$$\mathcal{P}_\varphi / \sim_\varphi = \{[\varphi]\}.$$

Thus the quotient forgets how the theorem was proved and remembers only the verified theoremhood represented by the successful class.

Proof. Every $P \in \mathcal{P}_\varphi$ pulls back by definition to a verified proof of φ from Γ. Hence every pair $P, Q \in \mathcal{P}_\varphi$ satisfies $P \sim_\varphi Q$. The restricted relation is therefore the universal relation on $\mathcal{P}_\varphi$, which is reflexive, symmetric, and transitive. If the set is nonempty, its quotient has exactly one class. □

This is the coarsest logical quotient for a fixed theorem. A short proof, a long proof, a proof discovered by BFS, a proof discovered by DATA, and any other verifier-accepted proof of the same φ are different representatives of the same logical class $[\varphi]$. The route still matters computationally, but not for the bare fact $\Gamma \vdash_F \varphi$.

Corollary 6.3 (Proof Horizon as minimum representative cost)**.** *Let $w : E \to \mathbb{Q}_{\geq 0}$ be the declared exact rational edge-cost map. Define the class cost by*

$$H_w(\Gamma, \varphi) = \inf_{P \in [\varphi]} w(P).$$

Whenever the presentation guarantees that the infimum is attained (for example, in a finite maze, an effectively proper bounded layer, or the unit-cost proof-length setting),

$$H_w(\Gamma, \varphi) = \min_{P \in [\varphi]} w(P).$$

Thus the weighted Proof Horizon is naturally a property of the entire theoremhood class rather than of any one chosen proof.

In the unit-cost normalization

$$w(e) = 1 \quad \text{for every inference edge } e,$$

we obtain

$$w(P) = |P|$$

and hence, whenever $\Gamma \vdash_F \varphi$,

$$H_1(\Gamma, \varphi) = \min_{P \in [\varphi]} |P|,$$

the shortest proof length. This recovers the unit-weight BFS/Proof-Horizon picture in quotient language: the theorem is the equivalence class, a concrete proof is a representative, and the Horizon is the cost of the cheapest representative.

Caution 6.4 (Logical quotient versus operational compression)*.* The theoremhood quotient is conceptually strong but does not by itself supply a search shortcut. To recognize that an unexplored tour belongs to $[\varphi]$ is already to establish that its pullback proves φ. Operational compression therefore requires finer effectively recognizable quotients or canonicalizations—for example, commuting independent inference steps, verified DAG identities, syntactic normal forms, or other verifier-respecting reductions that preserve the chosen reachability and cost objective. The theoremhood quotient supplies the semantic endpoint against which those finer reductions can be compared.

7 Purpose and scope

The motivating observation is that several strands of the earlier program fit into one diagram:

$$\text{search dynamics} \longrightarrow \text{cumulative verified history} \longrightarrow \text{SIC} \longrightarrow \begin{cases} \text{ATP}, \\ \text{ALD}, \\ \text{AMLD}. \end{cases}$$

A rich implementation such as DATA(x) is then a special case. The general mathematics should not depend on the names Professor, Counselor, Picard, Dreamer, Horizon, Prover, Refuter, or

Independence. Those modules matter operationally, but the structural results concern any verifier-backed staged search process satisfying the stated hypotheses.

The central computational premise is equally modest. In many theorem-proving or conjecture-settling instances, a valid certificate and hence a terminal state exists mathematically, while available hardware may not reach it. The principal problem is therefore not fixed-point existence but *computational accessibility*: how to shorten the hitting time, reduce the number of states that must be examined, compress repeated structure, and spend resources on branches that are more likely to reach a certificate.

8 General verified transition systems

Definition 8.1 (Verified transition system)**.** *A verified transition system is a tuple*

$$\mathfrak{M} = (X, \mathcal{F}, \pi, V, \mathcal{R})$$

where X is a set of complete instantaneous states, $\mathcal{F} = \{F_j : X \to X\}_{j \in J}$ is a family of admissible state transformations, π is a controller or scheduling policy selecting transformations, V is an exact verification gate for mathematical deposits, and $\mathcal{R}$ is the space of typed verified records.

A run is a sequence

$$x_0, x_1, x_2, \ldots, \qquad x_{n+1} = F_{j_n}(x_n),$$

with j_n selected by π or by an allowed nondeterministic/stochastic rule. The state x_n may include queues, heuristic parameters, random seeds, learned representations, resource allocations, untrusted proposals, checkpoints, and verified mathematical memory.

Caution 8.2 (The complete state need not be monotone). Nothing requires $x_n \subseteq x_{n+1}$ or any analogous order relation. A practical searcher may discard a queue, restore a checkpoint, lower a beam width, change a representation, or restart. Monotonicity will be imposed only on the verified-history projection.

9 The cumulative history of verified information

This construction is the bridge from general search dynamics to SICs.

Definition 9.1 (Verified stage output)**.** *Let V_n denote the set of mathematical records newly accepted by the independent verifier during the transition into stage n. Set $V_1 = \varnothing$ by convention, so that stage 1 is exactly the input stage. Depending on the machine, later V_n may include formulas, proved lemmas, proof certificates, refutation certificates, independence or dependence certificates, model witnesses, provenance data, formally checked derived rules, or other typed mathematical objects.*

Definition 9.2 (Cumulative history of verified information)**.** *For input Γ and stage $n \geq 1$, define*

$$C_{\mathfrak{M}}(\Gamma, n) = \Gamma \cup \bigcup_{k=1}^{n} V_k.$$

In particular, $C_{\mathfrak{M}}(\Gamma, 1) = \Gamma$. This is the cumulative history of verified information *through stage n.*

The phrase is meant literally. $C_{\mathfrak{M}}(\Gamma, n)$ is not a snapshot of everything the machine currently believes, proposes, scores, or searches. It is the permanent ledger of what has crossed the verification boundary. Search states may be abandoned; verified mathematics is not silently revoked.

Proposition 9.3 (Cumulative-history monotonicity)**.** *For every run and every n,*

$$C_{\mathfrak{M}}(\Gamma, n) \subseteq C_{\mathfrak{M}}(\Gamma, n+1).$$

Proof. By definition,

$$C_{\mathfrak{M}}(\Gamma, n+1) = C_{\mathfrak{M}}(\Gamma, n) \cup V_{n+1}.$$

□

Remark 9.4 (Projection, not necessarily quotient dynamics)*.* The cumulative history is a monotone observation of a possibly nonmonotone machine. Two complete states can have the same cumulative verified history while differing in queues, resources, or controller state and therefore having different futures. Thus the cumulative history need not be Markov-sufficient. It is a canonical SIC state even when it is not by itself a deterministic dynamical state.

10 The cumulative-history SIC theorem

The SIC framework used in *Depths of a Simulation* consists of a stage-state map C and halting map H with an initial state, monotone accumulation, persistent halting, and freezing after halting [14].

Definition 10.1 (Certificate halting)**.** *Fix a target condition $Z \subseteq \mathcal{R}$ of accepted terminal records. Define*

$$H_{\mathfrak{M}}(\Gamma, n) = 1 \quad \Longleftrightarrow \quad C_{\mathfrak{M}}(\Gamma, n) \cap Z \neq \varnothing.$$

After the first such stage, the operational machine may halt. For SIC analysis, extend the halted computation constantly thereafter.

Theorem 10.2 (Cumulative-History SIC Theorem)**.** *Suppose verified records are persistent and the computation is extended constantly after its first accepted terminal certificate. Then*

$$M_{\mathfrak{M}} = (C_{\mathfrak{M}}, H_{\mathfrak{M}})$$

is a semi-ideal computer on the chosen verified-record universe.

Proof. The initial condition is fixed by construction. Monotonicity is the previous proposition. If $H_{\mathfrak{M}}(\Gamma, n) = 1$, then the accepted terminal record remains in every later cumulative history, so halting persists. By the constant extension convention, the cumulative state is frozen after the operational halt. □

11 When the induced SIC is an ATP, ALD, or AMLD

11.1 ATP specialization

If the verified records are formulas and proof certificates in a frozen formal system, and the machine halts on target φ exactly when a verifier-accepted proof of φ is in the cumulative history, then the induced SIC is a target-search ATP once the usual soundness and eventual-success conditions are imposed. The internal search machinery can be arbitrarily elaborate; the SIC sees only the monotone verified ledger.

11.2 ALD specialization

For a target sentence φ over theory G, use typed terminal certificate classes

$$P : G \vdash \varphi, \qquad R : G \vdash \neg\varphi,$$
$$I : \mathrm{Ind}_G(\varphi), \quad \bot : \text{certified inconsistency of the base presentation.}$$

An ALD is then a typed cumulative-history SIC whose halting condition is the appearance of an accepted supported settlement certificate.

11.3 AMLD specialization

At the metalogical level, define

$$\mathrm{Dep}_G(\varphi) := (G \vdash \varphi) \vee (G \vdash \neg\varphi), \qquad \mathrm{Ind}_G(\varphi) := \neg\,\mathrm{Dep}_G(\varphi),$$

and type every meta-certificate by an explicit metatheory. An AMLD is the same SIC-lift principle applied one logical level higher. Timeout, stagnation, and budget exhaustion remain UNKNOWN unless a certificate language explicitly proves otherwise.

12 Halting and absorbing fixed points

Let $T : X \to X$ be a deterministic macro-update for a complete-state model and let $\mathcal{S} \subseteq X$ be the certified settlement states. Extend terminal states as absorbing:

$$x \in \mathcal{S} \Longrightarrow T(x) = x.$$

If every nonterminal complete state changes under T, then

$$\mathrm{Fix}(T) = \mathcal{S}.$$

This is the fixed-point settlement pattern proved for DATA 3.0 under an explicit round-counter hypothesis [19].

Proposition 12.1 (Fixed-Point/Halting Correspondence)**.** *Assume* $\mathrm{Fix}(T) = \mathcal{S}$*, and define* $H(\Gamma, n) = 1$ *exactly when* $x_n \in \mathcal{S}$*. Then*

$$H(\Gamma, n) = 1 \quad \Longleftrightarrow \quad x_n \in \mathrm{Fix}(T).$$

Caution 12.2 (Existence is not reachability)*.* The identity $\mathrm{Fix}(T) = \mathcal{S}$ characterizes terminal states; it does not imply that a given orbit reaches one. A fixed point can exist while its hitting time exceeds every practical hardware, memory, or wall-clock budget. The search problem is therefore to reduce the cost of reaching a fixed point, not merely to prove that fixed points exist.

13 Control AMLD theory: from static compassing to feedback settlement

The preceding sections separate logical authority from search policy: the verifier alone decides whether a certificate is accepted, while learned or hand-designed components decide where computational effort is spent. The next step is to make that policy itself a mathematical object. A *control AMLD* is an AMLD equipped with a feedback controller whose actions allocate search effort among proof, refutation, independence, representation change, and meta-level reasoning while preserving conservative certificate semantics.

13.1 State, actions, and the control objective

Let X_t denote the complete unsettled search state at control epoch t. It may contain the typed verified bank, the $P/R/I$ frontiers, learned compass features, resource counters, current incumbent certificates, and meta-level information. Let the action u_t encode a resource allocation or search intervention, for example

$$u_t = (r_P, r_R, r_I, r_M, a_{\text{repr}}, a_{\text{bank}}), \qquad r_i \geq 0, \qquad \sum_i r_i \leq R_t.$$

The controlled transition law may be stochastic,

$$X_{t+1} \sim K(\,\cdot \mid X_t, u_t\,),$$

while the settlement set $\mathcal{S}$ remains verifier-defined and absorbing. The controller does not certify theoremhood, refutability, or independence; it chooses computation intended to reach one of the accepted certificate classes sooner.

A basic finite-horizon objective is

$$J_\pi(x; B) = \Pr_x^\pi(\tau_{\mathcal{S}} \leq B),$$

and a cost objective is

$$V_\pi(x) = \mathbb{E}_x^\pi\left[\sum_{t<\tau_{\mathcal{S}}} c(X_t, u_t)\right],$$

with exact rational transaction costs available when desired. On a finite controlled theorem maze, standard Bellman recursion therefore supplies an exact benchmark value function against which a learned compass or scheduler can be compared. This makes control AMLD theory a bridge between the rational theorem-maze layer and the learned navigation layer rather than a replacement for either.

13.2 Abel coordinates and control-Lyapunov coordinates

The functional-equation viewpoint suggests searching for a scalar coordinate A in which successful progress is approximately translation:

$$A(Tx) - A(x) \approx 1.$$

Equivalently, with $V = -A$ up to an additive constant, useful actions should make a settlement-distance surrogate decrease. The relevant empirical quantity is not merely whether A correlates with proof depth, but whether the *executed controlled increments*

$$\Delta A_t = A(X_{t+1}) - A(X_t)$$

have positive drift and tolerable variance on held-out trajectories. A natural diagnostic is the Abel residual

$$\epsilon_A = \mathbb{E}\big|A(X_{t+1}) - A(X_t) - 1\big|.$$

This is stricter than a static rank metric: a model can rank proof-DAG nodes well yet choose an off-DAG successor and immediately destroy the trajectory.

Theorem 13.1 (Abel-drift settlement bound)**.** *Let $D(X_t) \geq 0$ before settlement and suppose that, for some $\mu > 0$,*

$$\mathbb{E}[D(X_{t+1}) - D(X_t) \mid \mathcal{F}_t] \leq -\mu$$

whenever $X_t \notin \mathcal{S}$. Under the standard optional-stopping integrability conditions, if τ is first settlement time, then

$$\mathbb{E}[\tau] \leq \frac{D(X_0)}{\mu}.$$

Proof. The stopped process $M_t = D(X_{t\wedge\tau}) + \mu(t \wedge \tau)$ is a supermartingale. Hence

$$\mathbb{E}[D(X_{t\wedge\tau})] + \mu\mathbb{E}[t \wedge \tau] \leq D(X_0).$$

Discard the nonnegative first term and pass to the limit under the stated integrability hypotheses. □

Theorem 13.2 (Finite-budget variance penalty)**.** *Assume progress increments have mean $\mu > 0$ and variance σ^2, with independence or martingale-difference hypotheses sufficient for additive variance. If cumulative progress D is required and $B\mu > D$, then Cantelli's inequality gives*

$$\Pr\left(\sum_{t=1}^{B} \Delta A_t < D\right) \leq \frac{B\sigma^2}{B\sigma^2 + (B\mu - D)^2}.$$

Thus a control signature should include variance as well as drift. A policy with slightly smaller mean progress but much smaller variance may be preferable under a hard proof budget.

Theorem 13.3 (Robust Lyapunov control under compass error)**.** *Suppose a true control-Lyapunov quantity $V \geq 0$ admits an ideal action with expected one-step decrease at least $\alpha > 0$. If the learned action loses at most ϵ expected decrease relative to the ideal action at each unsettled state, then*

$$\mathbb{E}[V(X_{t+1}) - V(X_t) \mid X_t] \leq -(\alpha - \epsilon).$$

If $\epsilon < \alpha$, the same drift argument yields

$$\mathbb{E}[\tau] \leq \frac{V(X_0)}{\alpha - \epsilon}.$$

This result gives a useful interpretation of compass error: the relevant question is whether approximation regret is smaller than the available control margin, not whether the learned distance is numerically exact.

13.3 P/R/I control and resource allocation

Equal-turn scheduling is a strong neutral baseline because it prevents starvation. Once agent-specific short-horizon response curves can be estimated, however, a feedback controller can ask a more precise question: where does the next unit of computation have the highest marginal settlement value?

Theorem 13.4 (Concave resource water filling)**.** *Suppose m AMLD agents receive resources $r_i \geq 0$ with $\sum_i r_i = R$, and agent i contributes an increasing differentiable concave short-horizon settlement hazard $h_i(r_i)$. Any maximizer of*

$$H(r) = \sum_i h_i(r_i)$$

satisfies the KKT conditions

$$h_i'(r_i) = \lambda$$

for active agents, while inactive agents satisfy $h_i'(0) \leq \lambda$.

The theorem does not establish that real $P/R/I$ response curves are additive or concave. It states what optimal allocation looks like if those measurable assumptions are approximately true over a control epoch. This yields a direct experiment: estimate h_P, h_R, h_I on sealed ALD/NOTALD tasks, compare water filling with equal-turn scheduling, and test whether a low-dimensional switching surface predicts reallocations.

13.4 Five deliberately open control-AMLD conjectures

The following are research targets rather than established properties of theorem search.

C1. Low-dimensional performance signature. A small vector of quantities such as global discrimination, distance-order quality, local tunneling quality, drift, variance, and Abel residual predicts held-out settlement probability and effort across broad theorem families.

C2. Emergent Abel coordinate. With sufficient true proof-DAG training, a scalar A emerges for which successful controlled transitions have an approximately stationary residual $A(Tx) - A(x) - c$ on unseen theorem families.

C3. Control-Lyapunov equivalence. On a broad finitely branching class, highly reliable settlement guidance is essentially equivalent to the existence of a learnable scalar or low-dimensional control-Lyapunov representation.

C4. P/R/I switching surfaces. With a shared verified bank and online hazard estimates, near-optimal resource control can be represented by relatively simple switching surfaces rather than long-memory chaotic alternation.

C5. Fast-global/slow-local learning. Compass learning has at least two distinct scales: global proof-geometry discrimination matures relatively early, while local direct-parent tunneling requires substantially more true DAG data.

14 Our settlement-compass experiments

The experimental program in the public ATP repository evolved in stages. Experiments 001 and 002 established the navigation-diagnostic machinery and small-DAG comparisons. Experiment 003 introduced shell-size comparisons. Those early studies were useful engineering and exploratory work, but changing training histories or candidate construction could confound a causal interpretation of training-set size. Experiment 004 was therefore redesigned as the primary clean learning-curve experiment. Experiment 005 then began the more demanding transition from static DAG ranking to executed control trajectories and Abel/Bellman diagnostics.

The public result directory is

https://github.com/btenneson/ATP/tree/main/optimizations/settlement_compass/results.

14.1 Experiment 004: frozen nested shells and the MAX "stupid case"

One eligible proof-DAG universe was split once into a sealed twenty-target test set and a training pool. Nested shells were taken from one frozen ordering,

$$S_{10} \subset S_{20} \subset S_{40} \subset S_{80} \subset S_{160} \subset S_{\max},$$

with every shell trained from scratch under the same feature coordinates, negative samples, test candidate pools, seeds, and model class. The deliberately oversized MAX condition used all 3,612 eligible remaining true training roots. This was designed to reveal whether additional true DAG data still mattered after the ordinary shells appeared to saturate.

Training roots	AUC	Distance ρ	MAE	P@10	Median direct-parent rank
10	0.8353	0.2398	2.5042	0.450	569
20	0.8650	0.2784	2.4310	0.415	237.5
40	0.8761	0.3149	2.5080	0.490	219
80	0.8876	0.3318	2.7019	0.570	251
160	0.8880	0.3427	2.8086	0.555	199
3612 (MAX)	0.8824	0.3483	2.3993	0.670	57

Table 3: Experiment 004 aggregate results on one sealed twenty-target set. The experiment is a retrospective proof-DAG navigation diagnostic, not an end-to-end ATP race.

The global classifier essentially stops improving in AUC by the 80–160 region, whereas local navigation remains sensitive to the very large corpus. At MAX, precision@10 reaches 0.670, median first direct-parent rank falls to 57, and the compass beats random direct-parent ordering on 17/20 targets, versus 12/20 at the 160 shell. The MAX classifier used roughly 1.98 million classifier examples and 1.37 million regression examples.

14.2 Illustrative learning-curve fits

With only six aggregate training conditions, curve fitting must remain descriptive. The purpose is to estimate plausible scales and design the next shells, not to declare a universal learning law. A simple saturation model is

$$y(n) = y_\infty - be^{-n/\tau}.$$

Least-squares fits to the current aggregate values give:

Metric and fitting range	y_∞	τ	R^2	Interpretation
AUC, shells 10–160	0.8868	14.3	0.982	Very fast global-discrimination saturation; MAX AUC 0.8824 is slightly below the fitted plateau.
Distance Spearman, shells 10–160	0.3397	21.8	0.996	Global distance ordering also matures quickly; MAX 0.3483 modestly exceeds the five-shell extrapolation.
Precision@10, all six conditions	0.6643	137.6	0.893	A substantially slower local/top-k scale is compatible with the data.

Table 4: Illustrative one-timescale fits. The fitted τ values are model parameters, not proven sample-complexity constants.

Under this model the P@10 timescale is about 9.6 times the AUC timescale. That is quantitatively consistent with the fast-global/slow-local hypothesis, but the interval $160 \to 3612$ is too sparse to identify a genuine local exponential scale. A different low-parameter model also fits the local statistic reasonably well: on all six points, a log–log regression gives approximately

$$\mathrm{MedRank}_{\mathrm{parent}}(n) \approx 928\, n^{-0.335},$$

with $R^2 \approx 0.889$ in log space. Because both saturation and power-law descriptions remain plausible at this resolution, the correct conclusion is experimental: add intermediate shells, not choose a favorite curve prematurely.

The highest-value interpolation points are therefore roughly

$$n \in \{320, 640, 1280, 2560\},$$

with the original twenty test targets and every other frozen experimental choice unchanged. Model comparison should be done on predictive error, preferably by withholding one or more shell conditions or, better, by repeating the entire shell protocol on unrelated theorem families.

14.3 Experiment 005: the controller is harder than the compass

Experiment 005 extends the program to "interpolation + retrospective executed DAG control trajectories" and explicitly remains *not* a live ATP race. The trajectory diagnostics begin measuring quantities required by the control theorems: learned and true drift, increment variance, Abel residual, Bellman-value error, hitting time, and trajectory settlement.

One currently recorded aggregate at the 160-DAG shell is particularly instructive. Across 100 retrospective trajectories, the learned progress coordinate had estimated drift

$$\widehat{\mu} \approx 0.0526,$$

learned increment variance about 0.659, mean learned Abel residual about 1.016, and Bellman-value MAE about 2.156 on executed states, yet the recorded trajectory success rate was

$$0/100.$$

This is not a contradiction of the drift theorem: its hypotheses require a suitable uniform conditional decrease toward the actual settlement set, not merely a positive aggregate statistic on a mixture of transitions. The trajectories frequently terminate by selecting distractor/off-DAG states. The result is therefore scientifically useful: it shows exactly why control-AMLD evaluation must measure closed-loop trajectories rather than infer control quality from AUC, Spearman correlation, or static top-k ranking alone.

14.4 What the experiments currently support

The present evidence is strongest for three claims of limited scope:

1. more true proof-DAG training measurably improves several held-out navigation statistics, with diminishing returns in global AUC;
2. global discrimination and local tunneling need not saturate at the same training size;
3. a static compass can look useful while a naive closed-loop controller still fails, so successor selection, off-DAG robustness, and control stability are independent research problems.

The experiments do *not* yet establish an end-to-end AMLD speedup, an optimal P/R/I resource policy, an Abel coordinate with stationary residual, or a universal two-timescale law.

14.5 Next control experiments

The next experiments should be deliberately causal and controller-facing:

(i) fill the $160 \to 3612$ interval with frozen intermediate shells and refit one-, two-, and simple power-law learning curves;

(ii) log complete executed trajectories so that μ, σ^2, ϵ_A, failure mode, and hitting time are measured per theorem rather than only in aggregate;

(iii) on finite proof DAGs, compute the exact Bellman value function and compare the learned controller's action regret with the true one-step optimal action;

(iv) run equal-turn versus hazard-water-filling $P/R/I$ schedules on sealed NOTALD controls before interpreting any gain on genuine independence problems;

(v) include deliberate detour mazes where every successful route must temporarily increase a naive distance, testing whether scalar Lyapunov control is sufficient or whether vector/history-valued state is required;

(vi) retain an exhaustive/fair background search so that aggressive control changes order and resource allocation without silently deleting certificate-complete coverage.

The control-AMLD program therefore turns a broad slogan—"make the search more intelligent"—into measurable quantities. The central empirical object is no longer just a classifier score; it is the distribution of verifier-respecting settlement trajectories under feedback.

15 Prior synthetic benchmark evidence, with disclaimers

The technical companion to the Proof Horizon reported an $L^* = 100$ synthetic dry tournament on 20 independently seeded finite "oceans." Each ocean had an externally established shortest successful route of exactly 100 edges. General searchers were not told $L^* = 100$ and were not given the hidden shortest route. The stopping rule was first verifier-accepted proof [17].

System	Finished	Optimum first	Median L	Median E	Median time (ms)
Depths-F specialized control	20/20	20/20	100	100	0.002
BFS	20/20	20/20	100	10928.5	3.013
DFS	20/20	4/20	115	1747.5	0.487
Beam-64	19/20	16/20	100	5546.5	2.664
Data-ATP Hilbert-only ablation	20/20	6/20	115	1317.5	7.115
Data-ATP full dry stand-in	20/20	11/20	100	375	30.747
DATA 2.0.1 fast wing	20/20	7/20	115	121.5	28.570
DATA 2.0.1 Horizon certifier	20/20	20/20 certified	100	10928.5	3.037

Table 5: Previously reported synthetic dry results at $L^* = 100$; reproduced here only as architectural evidence and benchmark context, not as a production-performance claim.

Required disclaimer. These numbers are *not* production benchmarks of the full Data-ATP or DATA repositories, do not establish superiority on natural mathematics, and should not be read as a universal speedup theorem. They are executable stand-ins designed to separate intrinsic path length from navigation overhead and to test measurement, failure reporting, and architecture. In that dry setting, the fuller Data-ATP stand-in used far fewer median expansions than BFS while the separate Horizon certifier recovered/certified the known optimum at exhaustive-search cost. The result motivates, but does not validate, the larger search-reduction program [17].

16 A proposed rational-weight theorem-maze benchmark

The next benchmark should test the graph-theoretic claim directly and should use exact rational weights rather than floating-point surrogates.

Definition 16.1 (Rational Ocean benchmark family)**.** *A benchmark instance is a finite or effectively bounded directed theorem maze with:*

1. *a planted or independently certified minimum settlement cost $L_w^* \in \mathbb{Q}_{\geq 0}$;*
2. *exact edge weights a/b stored in reduced rational form;*
3. *a frozen verifier and certificate language;*
4. *adjustable width, dead-end density, alternative successful routes, repeated substructures, and misleading locality;*
5. *optional typed exits for P, R, I, and $\perp$;*
6. *a hidden shortest route unavailable to general searchers.*

The benchmark should compare at least a rational Dijkstra baseline, rational A* with frozen admissible heuristics, BFS on unit-weight controls, beam/best-first variants, portfolio search, shared-bank ablations, and the relevant ATP/ALD/AMLD architecture. Every addition must preserve exact certificate checking.

Primary measurements should include verified settlement rate and type, first-settlement cost, certified minimum settlement cost where attempted, node/edge expansions, wall-clock time, peak memory, verifier time, bank deposits and retrievals, restarts, and the gap

$$\Delta = \text{Cost(first verified settlement)} - L_w^*.$$

For minimum-cost experiments, report separately the cost of discovery and the cost of certification.

Benchmark disclaimer. A rational-weight synthetic maze is a controlled model of proof-search geometry, not a claim that real theorem libraries have the same distribution. Success is meaningful only if ranking survives held-out changes in width, decoys, weight denominators, representation, and eventually frozen natural formal-mathematics benchmarks.

17 Reflective ATPs: proving things about the prover

Once the cumulative-history SIC satisfies ATP conditions, the induced ATP can itself be given a formal target language containing an effective code of a frozen machine M.

Let $M^{\#}$ be a coded or tagged copy of the machine, in the spirit of the tagging constructions in the author's *Depths* line [14]. The meta-ATP can then search for derivations of statements such as

"every accepted P-certificate of $M^{\#}$ verifies,"

"the terminal states of $M^{\#}$ are absorbing,"

"this scheduler is fair on this finite benchmark,"

or bounded search facts such as

"no certificate of declared cost $< B$ occurs in this effectively finite layer."

If such a theorem is independently verified, it may be deposited in a typed meta-level bank and used by a controller as mathematical information about the search process.

Theorem 17.1 (Meta-ATP construction, schematic form)**.** *Suppose the cumulative-history lift of M is an effective ATP A_M, and the background formal theory effectively represents a frozen code $M^{\#}$ together with the relevant syntactic verification predicates. Then A_M can search for every derivable encoded property of $M^{\#}$ expressible in that theory. Any accepted result is a theorem of the background formal system, not an oracle statement about semantic truth.*

Caution 17.2 (No unrestricted self-certification)*.* This construction does not evade the classical incompleteness, undecidability, or undefinability barriers associated with Goedel, Church, and Tarski [20, 21, 22]. It is most credible for bounded, syntactic, relative, conditional, and verifier-checkable properties. A machine proving a coded local fact about its search is not the same thing as a machine proving its unrestricted global soundness.

18 DATA as a distinguished case

DATA 3.0 is a rich instance of the general framework. Its complete state includes object theory, target, metatheory, typed verified bank, multiple search frontiers, untrusted proposals, history, learned parameters, resource allocation, candidate/accepted certificates, settlement status, and a monotone round counter [19]. Its active subsystems are modeled as transformations whose compositions generate a transformation semigroup.

The general theory above interprets DATA in four simultaneous ways:

1. the instantaneous DATA process is a controlled transition system or IFS-like semigroup action;
2. its cumulative verified history is a SIC;
3. the P route specializes to an ATP, while the combined $P/R/I$ routes form an ALD and the nested dependence/independence route forms an AMLD;
4. the resulting theorem/settlement graph is a rationally weighted maze once exact rational transaction costs are declared.

The DATA-specific intelligence layer can therefore be understood as a collection of maze-navigation policies. Professor scores frontiers; Counselor challenges representations; Picard supplies feedback; Creativity and Dreamer generate candidate moves; Horizon certifies bounded regions; the verified bank adds reusable macro structure. None of these changes the verifier's semantics.

19 A search funnel for computational accessibility

The synthesis of the Horizon and Depths programs suggests the following funnel:

raw staged search dynamics
↓
cumulative verified-history SIC
↓
novelty / quotient / DAG reduction
↓
conservative compilation and shared verified landmarks
↓
verified incumbent bound B
↓
rational branch-and-bound / Dijkstra / A* / exact graph methods
↓
learned ranking, portfolios, reconnaissance, and resurvey
↓
certificate-bearing fixed point.

The strongest claim is not that the fixed point becomes easy. It is that each justified transformation can remove a class of states or routes that do not need to be searched for the declared objective.

20 Research conjectures and implementation questions

Conjecture 20.1 (Verified-history compression)**.** *On broad formal-mathematics families, searching a canonicalized graph of cumulative verified histories requires asymptotically or practically fewer expansions than searching raw transient histories under matched correctness constraints.*

Conjecture 20.2 (Compiled-landmark acceleration)**.** *A verified shared bank plus conservative macro compilation reduces rational weighted hitting time on families with repeated substructure, after retrieval and verification overhead are charged exactly.*

Conjecture 20.3 (Admissible learned lower bounds)**.** *For some nontrivial theorem families, a learned function can be calibrated into a valid rational lower bound $h(v)$ often enough to improve branch-and-bound or A* search while preserving exact minimum-cost guarantees.*

Conjecture 20.4 (Reflective search improvement)**.** *A meta-ATP can prove bounded or conditional facts about a frozen source ATP/ALD/AMLD that, when deposited into the verified meta-bank, measurably reduce later settlement cost without altering the verifier or certificate semantics.*

Key implementation questions include whether cumulative-history canonicalization is cheap enough to pay for itself; how to define rational edge costs that remain meaningful across machines; how much of proof-state equivalence can be checked exactly; whether macro compilation should affect search cost, native proof cost, or both; and which meta-theorems are strong enough to improve scheduling rather than merely restating architecture.

21 Conclusion

The central proposal is that theorem proving and conjecture settlement can be studied as exact rationally weighted maze solving over verifier-backed state graphs. The internal search process may be nonmonotone, stochastic, learned, or self-reflective, while its cumulative history of verified information forms a monotone SIC. Under additional target-search conditions that SIC is an ATP; with typed proof/refutation/independence certificates it becomes an ALD; one logical level higher it becomes an AMLD.

The graph-theoretic perspective matters because it imports a large body of route-finding ideas subject to explicit hypotheses. For finite nonnegative rational weights, Dijkstra-style minimum-cost search is exact. With admissible lower bounds, A*-style and branch-and-bound pruning can reduce the region below an incumbent. Novelty DAGs eliminate verified-state cycles, conservative compilation replaces repeated verified paths by macro edges, and shared lemma banks turn repeated proof structure into reusable landmarks. These reductions do not solve undecidability or guarantee practical access to every fixed point. They narrow the ocean in which the machine must search.

The further reflective possibility is that the induced ATP can itself reason about a coded source ATP, ALD, AMLD, or controller. If such meta-theorems can be verified and fed back into search, the machine does not merely learn statistically from its trajectories; it can sometimes use proved facts about its own frozen search architecture. DATA is a distinguished laboratory for this general theory, not its definition.

References

[1] Edward F. Moore. *The Shortest Path Through a Maze*. In *Proceedings of the International Symposium on the Theory of Switching, Part II*, pp. 285–292. Harvard University Press, 1959.

[2] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. https://doi.org/10.1007/BF01386390

[3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. https://doi.org/10.1109/TSSC.1968.300136

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, 4th ed. MIT Press, 2022. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

[5] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020. https://aima.cs.berkeley.edu/

[6] Robert W. Floyd. Algorithm 97: Shortest path. *Communications of the ACM*, 5(6):345, 1962. https://doi.org/10.1145/367766.368168

[7] Stephen Warshall. A theorem on Boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962. https://doi.org/10.1145/321105.321107

[8] Antti Valmari. A stubborn attack on state explosion. *Formal Methods in System Design*, 1(4):297–322, 1992. https://doi.org/10.1007/BF00709154

[9] Doron Peled. All from one, one for all: on model checking using representatives. In *Computer Aided Verification (CAV 1993)*, Lecture Notes in Computer Science 697, pp. 409–423. Springer, 1993. https://doi.org/10.1007/3-540-56922-7_34

[10] Alice Miller, Alastair F. Donaldson, and Muffy Calder. Symmetry in temporal logic model checking. *ACM Computing Surveys*, 38(3), 2006. https://doi.org/10.1145/1132960.1132962

[11] Richard E. Korf and Ariel Felner. Disjoint pattern database heuristics. *Artificial Intelligence*, 134(1–2):9–22, 2002. https://doi.org/10.1016/S0004-3702(01)00092-3

[12] Michel Gondran and Michel Minoux. *Graphs, Dioids and Semirings: New Models and Algorithms*. Springer, 2008. https://doi.org/10.1007/978-0-387-75450-5

[13] Brian Tenneson. *A Closed-Form Surjection from N onto Q*. Version 2.2, 2026. Live GitHub copy: https://github.com/btenneson/pub/blob/main/math.LO_Logic/A%20Closed-Form%20Surjection%20from%20N%20onto%20Q%20by%20Brian%20Tenneson%202.2.pdf

[14] Brian Tenneson. *Depths of the theorem-search program: graph-theoretic proof search, SICs, and automated theorem proving*. Public 2026 research manuscript in the *Depths* line; live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.LO_Logic_in_Computer_Science/Depths_of_Induction_ML_Guided_Proof_Search_and_Formalized_Self_Awareness_Version_47.pdf

[15] Brian Tenneson. *The Universal Proof-Horizon Operator: From MIU and the nth Well-Formed Formula to ZFC, Predator, and Automated Logical Deciders*. Version 1.0, August 2026. Live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.LO_Logic_in_Computer_Science/The_Universal_Proof-Horizon_Operator_by_Brian_Tenneson.pdf

[16] Brian Tenneson. *DATA 2.0.1: Proof-Horizon Architecture / From a Target Formula to a Verified Minimum-Cost Proof*. August 2026. Live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.LO_Logic_in_Computer_Science/DATA_2_0_1_Proof_Horizon_Architecture_FINAL.pdf

[17] Brian Tenneson. *A Technical Companion to My Next Generation Automated Theorem Prover*. 2026; includes the Proof-Horizon/Ocean synthetic benchmark material used here. Live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.LO_Logic_in_Computer_Science/A_Technical_Companion_to_My_Next_Generation_Automated_Theorem_Prover_v1.0.pdf

[18] Brian Tenneson. *Hilbert Space-Filling Curve Theorem Search*. Version 0.5.3, 2026. Live GitHub copy: https://github.com/btenneson/pub/blob/main/cs.LO_Logic_in_Computer_Science/Hilbert_Space_Filling_Curve_Theorem_Search_v0.5.3.pdf

[19] Brian Tenneson. *DATA 3.0: AMLD/IFS Semigroup Architecture, Settlement Horizons, and Conjecture-Settling Research Program*. August 2026. Live GitHub research library: https://github.com/btenneson/pub/tree/main/cs.LO_Logic_in_Computer_Science

[20] Kurt Goedel. Ueber formal unentscheidbare Saetze der Principia Mathematica und verwandter Systeme I. *Monatshefte fuer Mathematik und Physik*, 38:173–198, 1931. https://doi.org/10.1007/BF01700692

[21] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936. https://doi.org/10.2307/2371045

[22] Alfred Tarski. Der Wahrheitsbegriff in den formalisierten Sprachen. *Studia Philosophica*, 1:261–405, 1935.